

Document Extraction Platform

Schema-defined extraction of structured data from uploaded documents, with natural-language querying over both extracted fields and document content.

Stack: Vite + React · Node/Express · SQLite · LLM extraction — Audience: technical leadership & implementers

I Two Architecture Options

OPTION A

Modular Monolith

Recommended

One Node/Express process. Uploads enqueue a job row in SQLite; an in-process worker loop performs parsing, LLM extraction, and embedding. Zero extra infrastructure.

React SPA → Express API + in-process worker → SQLite (WAL) → LLM API

STRENGTHS

- Single deployable; trivial ops & local dev
- SQLite job table doubles as audit trail
- Fastest path to a working product

TRADE-OFFS

- LLM latency shares CPU/event loop with API
- Vertical scaling only
- A crash mid-job pauses all extraction

OPTION B

API + Separate Worker

API and extraction worker are separate Node processes sharing SQLite (WAL mode). Workers subscribe to job inserts via SQLite CDC (update hook / session extension off the WAL) instead of polling; the client receives progress over SSE.

React SPA → Express API ↔ SQLite (WAL) → CDC → Worker *N → LLM API

STRENGTHS

- Extraction throughput scales independently
- Retries & crash isolation per job
- API stays responsive under heavy ingestion
- CDC push means near-zero pickup latency, no idle poll load

TRADE-OFFS

- Two processes to deploy & monitor
- SQLite single-writer contention at high volume
- Beyond ~1 machine, forces Postgres + queue migration

Recommendation: start with Option A. Both options share the identical data model and API surface below — the split is purely operational, so A can graduate to B without breaking clients.

II System Layers

CLIENT	Schema Builder (columns: name · description · type) · Upload (schema selection required) · Document Grid (extracted values per schema) · Ask (plain-English query box)
API — EXPRESS	Auth Service (register/login against admin DB, issues JWT) · Tenant Router (user id → tenant DB) · Schema Service · Ingestion Service (multer upload, enqueues job in admin DB) · Query Service (NL → filters + semantic) · SSE progress
WORKER	Claim job (CDC push from admin DB job insert, opens claimed user's tenant DB by path) → Parse (pdf-parse · mammoth · Tesseract OCR fallback) → LLM Extract (schema → JSON tool call, per-field confidence) → Validate & Coerce → Chunk & Embed
STORAGE — SQLITE	Admin DB (users, cross-tenant extraction_jobs queue) · Tenant DB ×N — one SQLite file per user (schemas · documents · values · chunks) · FTS5 & sqlite-vec per tenant DB · originals on disk, namespaced by user id
EXTERNAL	LLM API (extraction + NL query parsing + answer synthesis) · Embedding API or local model (e.g. all-MiniLM via transformers.js)

III Data Model

Admin DB (users, jobs) + one SQLite file per user · WAL

users — admin database id PK · username UNIQUE · email UNIQUE · password_hash · tenant_db_path · created_at · last_login_at	document_schemas — user-defined templates (tenant DB) id PK · name UNIQUE · description · created_at · updated_at
schema_columns — fields to extract id PK · schema_id FK · name · description (guides the LLM) · data_type ('text' 'number' 'date' 'boolean' 'enum') · enum_options JSON · required · position	documents — uploaded files id PK · schema_id FK · filename · mime_type · storage_path · size_bytes · status ('uploaded' 'processing' 'extracted' 'failed') · uploaded_at
extracted_values — one row per column id PK · document_id FK · column_id FK · value_text · value_number REAL · value_date · confidence 0-1 · source_snippet	document_chunks — semantic index id PK · document_id FK · chunk_index · text (mirrored in FTS5) · embedding BLOB float32 (sqlite-vec)

extraction_jobs — admin database, shared queue

id PK · user_id FK → users · document_id (id in that user's DB) · status ('queued'|'running'|'done'|'failed') · attempts · max_attempts · error · started_at · finished_at

users 1—1 tenant database · 1—* extraction_jobs (admin DB) · document_schemas 1—* schema_columns · 1—* documents (in-tenant) · documents 1—* extracted_values · 1—* document_chunks · typed value columns make NL filters directly SQL-able.

IV API Endpoints

REST · JSON · multipart for upload

METHOD	PATH	PURPOSE
POST	/api/auth/register	Create user in admin DB; provisions a new tenant SQLite file
POST	/api/auth/login	Verify credentials against admin DB; returns session/JWT
POST	/api/schemas	Create schema with columns[] (name, description, data_type) — in caller's tenant DB
GET	/api/schemas	List schemas with column + document counts
GET	/api/schemas/:id	Schema detail incl. columns
PUT	/api/schemas/:id	Update; column changes versioned if documents exist
DELETE	/api/schemas/:id	409 if documents attached (or ?force=true cascades)
POST	/api/documents	multipart: file + schema_id (required) → 202 + job id
GET	/api/documents?schema_id=	List documents, filterable by schema & status
GET	/api/documents/:id	Metadata + status + extracted values
GET	/api/documents/:id/file	Download / preview the original
GET	/api/documents/:id/events	SSE stream: processing progress
POST	/api/documents/:id/reextract	Re-run extraction (schema edited, failure retry)
DELETE	/api/documents/:id	Remove file, values, chunks
POST	/api/query	{ question, schema_id? } → answer + matched documents + citations
GET	/api/queries	Query history with parsed-filter audit
GET	/api/admin/jobs	Admin-only: cross-tenant job queue status & failures

V User Flow

1. Define schema — user names a schema and adds columns, each with name, description, and data type. Description doubles as the extraction hint.
2. Upload document — upload requires selecting a schema; the API rejects files without schema_id. Returns 202 immediately.

3. Automatic extraction — worker parses the file, calls the LLM with the schema as a tool definition, validates types, stores values + chunks.
4. Review & correct — document grid shows extracted values with confidence; low-confidence cells flagged for one-click human correction.
5. Ask in English — “Invoices from Acme over \$10k in March” — the system answers with matching documents and cited snippets.

VI Sequence Flows

Upload & Extraction ASYNCHRONOUS · STATUS VIA SSE OR POLLING

1. Client — POST /api/documents (file + schema_id); API validates type/size and that the schema exists
2. API — stores the file on disk, inserts documents row (status: uploaded) + extraction_jobs row (queued) in one transaction; returns 202
3. CDC — admin DB insert into extraction_jobs (status: queued) is captured off the WAL and pushed to the worker pool — no polling
4. Worker — receives the change event, claims the job (status: running), extracts text — pdf-parse / mammoth, Tesseract OCR fallback for scans
5. Worker — calls LLM with the document text and schema columns as a JSON tool definition → typed values + confidence + source snippets
6. Worker — validates & coerces each value against data_type; writes extracted_values
7. Worker — chunks text (~500 tokens, overlap), embeds, writes document_chunks + FTS5 index; sets status: extracted
8. Client — receives completion over SSE; grid row fills in with extracted values

Natural-Language Query HYBRID · STRUCTURED FILTERS + SEMANTIC SEARCH

1. Client — POST /api/query { question: "invoices from Acme over \$10,000 due in March" }
2. API — sends question + available schemas/columns to the LLM → parsed intent JSON: structured filters + semantic terms
3. API — filters → SQL over extracted_values typed columns (value_number > 10000, value_date range...)
4. API — semantic terms → embed → sqlite-vec KNN over chunks, in parallel with FTS5 keyword match
5. API — merges result sets via reciprocal rank fusion; structured filters act as hard constraints
6. API — LLM composes a short answer citing matched documents and source snippets
7. Client — renders answer + document list; each citation links to the original file

*Examples: “invoices from Acme over \$10,000 due in March” → filter: vendor = Acme, amount > 10000, due_date ∈ March
· “contracts that mention early termination” → semantic search over chunks.*

Key decisions. LLM tool-calling (not regex/NER) for extraction — schemas are user-defined at runtime, so extraction must be zero-shot. Typed value columns make NL filters cheap SQL. Multi-tenant isolation via one SQLite file per user, with a shared admin database holding accounts and the cross-tenant extraction job queue. Workers pick up jobs via SQLite CDC (capturing inserts off the WAL) rather than polling, so pickup latency stays flat as job volume grows. sqlite-vec + FTS5 keep each tenant file self-contained until scale forces Postgres + pgvector — the API contract is unchanged by that migration. v1.2 · July 2026